FoveaWAM: Role-Aware 4D World–Action Modeling for Dynamic Manipulation

Anonymous CVPR submission

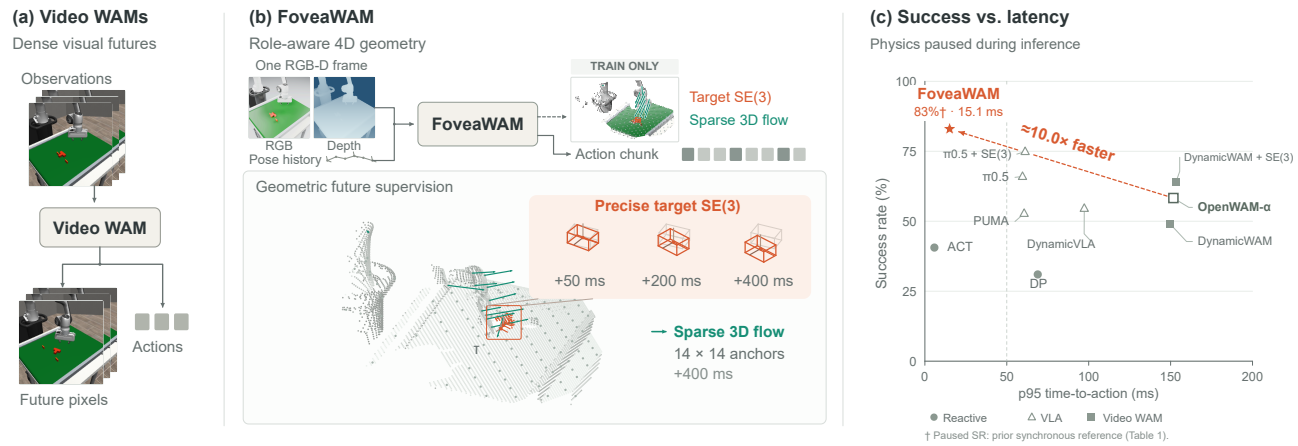


Figure 1. **FoveaWAM allocates prediction precision by task role.** (a) Reference recordings illustrate dense video-future prediction and action outputs. (b) One RGB-D frame and pose history condition FoveaWAM, with training-only target SE(3) and sparse 3D flow supervision; action chunks are produced at deployment. The RGB illustration is a Blender rendering of the recorded state; depth and point clouds use recorded observations. Future geometry shows ground-truth supervision. Flow is sampled on a 14×14 grid; arrows show 400 ms displacements enlarged $3\times$. The lower pose glyphs show the target at $+50$, $+200$, and $+400$ ms. (c) Table 1 Paused success (physics paused during inference) versus native p95 time-to-action. The dashed line marks 50 ms. The FoveaWAM star uses the reported 83%† and measured 15.1 ms, giving $\approx 10.0\times$ speedup over OpenWAM-α (151.7 ms). †The Paused value follows the prior synchronous reference selected in Table 1.

Abstract

Manipulating moving objects requires actions that fit the scene when they execute, not when the image was captured, yet every millisecond spent inferring the scene's motion lets the target move further. World–action models forecast future images or latents at uniform fidelity, spending time on appearance while leaving the target's metric pose implicit; target-centric predictors are fast but ignore the surroundings that constrain or drive the motion. We observe that the parts of a dynamic scene need different precision: the target's pose decides contact to within millimeters, the bodies around it matter only through the clearance they leave and the motion they impart to the target, and the remaining background does not matter. We present FoveaWAM, a world–action model built on this role-aware 4D structure. It encodes the target's motion as a metric SE(3) pose history, so one RGB-D frame suffices as input. It learns to forecast the target's future SE(3) trajectory precisely and the whole scene coarsely as sparse 3D flow, with forecasts used only in training. With compiled BF16 inference, FoveaWAM has a p95 time-to-action of 15.1 ms on one RTX 5090, $10.0\times$ *faster than OpenWAM-*α*. We also introduce OnTimeBench, an open-source benchmark of 14 simulated dynamic manipulation tasks in which the scene keeps moving throughout each policy's inference. FoveaWAM achieves 76.0% success on OnTimeBench, 16.4 points above the strongest baseline, also leads on 24 tasks of the DOMINO benchmark, and succeeds in [XX]% of trials on six real-robot tasks with diverse target motion.*

1. Introduction

Learning-based manipulation has advanced rapidly, from diffusion policies to vision–language–action (VLA) and world–action models (WAMs) [5, 6, 11, 19, 38, 44, 58], yet most of this progress assumes that objects stay still unless the robot moves them. In factories, kitchens, and human–robot handovers, objects keep moving while the robot perceives, computes, and acts. In such scenes, an action must fit the

scene at the moment it executes, not the moment its observation was captured, so a policy has to know how the scene moves. That knowledge must itself be inferred, and inference takes time during which the scene keeps moving: the longer a policy computes to anticipate the motion, the further ahead it must anticipate.

Recent dynamic policies add a memory of past motion, typically image-space cues such as optical flow [13, 25], and a forecast of the future [25, 34, 51]. Video-based WAMs forecast whole future frames or latents at uniform fidelity, spending much of their computation on visual detail [38, 44]. Compact future codes [26] reduce this cost and metric 3D scene dynamics [49] add geometry, but both still treat the whole scene uniformly. In each case, the quantity that decides contact, the target's metric pose at execution time, stays implicit. Target-centric predictors and dynamic VLAs with object-level forecasts sit at the other end [13, 16]: they are fast but anticipate only the target, leaving out the surroundings that can block or redirect its motion. Neither approach allocates precision according to what contact requires from each part of the scene. This raises a central question: what must a policy know about a moving scene, and how precisely, to make contact while keeping inference latency low?

Our key observation is that a dynamic policy needs to know very little precisely (Fig. 1). Contact demands millimeter precision in the target's pose at execution time; the surroundings matter only through the clearance they leave and the motion they impart to the target, so they need only be known coarsely. Building on this observation, we introduce FoveaWAM, a world–action model that learns a role-aware metric 4D representation, precise where contact demands it (Fig. 2). To supply motion information without encoding past image frames, FoveaWAM pairs a single RGB-D frame with a short history of the target's metric SE(3) poses. A world stream learns the representation by forecasting the target's future SE(3) trajectory and the motion of the whole scene as a sparse 3D flow. An action stream reads the world stream's scene and summary tokens to generate action chunks, with explicit forecasting confined to training [48]. FoveaWAM thus combines precise anticipation of target motion with low inference latency, limiting how much the scene changes between observation and action.

We build OnTimeBench, an open-source benchmark of 14 simulated dynamic manipulation tasks in which the scene keeps moving throughout each policy's inference, so every method pays for its own latency. FoveaWAM achieves 76.0% success on OnTimeBench, 16.4 points above the strongest baseline, at a p95 latency of 15.1 ms with compiled BF16 inference, $10.0\times$ faster than OpenWAM-α. It also leads on 24 DOMINO tasks [13] and succeeds in [XX]% of trials on six real-robot tasks, against [XX]% for the strongest baseline. Controlled ablations test the contributions of target-motion history and future supervision, and compare role-aware with uniform-fidelity forecasts to examine the allocation of precision.

Our contributions are threefold:

- We show that a dynamic policy needs millimeter precision only in the target's pose at execution time, and instantiate this principle as role-aware 4D modeling: a precise metric SE(3) trajectory for the target and a coarse sparse 3D flow for the whole scene.
- We develop FoveaWAM, a world–action model that combines metric target-pose histories with role-aware predictive supervision for low-latency action generation.
- We introduce OnTimeBench, an open-source dynamic manipulation benchmark that charges each policy its own inference latency, and demonstrate gains over strong baselines on OnTimeBench, 24 DOMINO tasks, and six real-robot tasks.

2. Related Work

Dynamic manipulation. Compensating for the time between sensing and acting is a classical problem, from control with dead time [31] to catching systems that track the target with Kalman-type filters and plan an interception [1–3, 28]; such systems are precise but engineered for each object and task. Learned policies have recently been brought to moving scenes [42]. PUMA reads historical optical flow and is supervised with future object features [13], and DynamicWAM conditions a WAM on flow and kinematic descriptors and executes it with real-time chunking [25]. Both infer past motion from image-space cues, whereas FoveaWAM takes the target's metric SE(3) pose history as input and observes only the current RGB-D frame. Other dynamic policies forecast salient tokens [34], future images [52], compact future codes [26], or 3D scene dynamics [49] and condition their actions on these forecasts at test time. DynaWM adapts a base VLA's action chunks for dynamic manipulation [10], and DynamicManip synthesizes dynamic demonstrations from a static one [21]. Execution schemes reduce or hide latency by overlapping inference with execution [30, 42], keeping consecutive chunks consistent [7, 23], or conditioning on the execution time [35]. They decide when actions are computed and applied, not what the policy represents. DOMINO [13] and PhysMani-Bench [49] provide diverse dynamic-manipulation tasks, but their released default evaluation pipelines advance the simulation only after a policy has returned its prediction. OnTimeBench advances the scene during inference and charges each method its measured latency.

Predictive supervision in WAMs. World–action models (WAMs) generate future video and actions in one network [20, 38, 44, 56], building on video prediction as a policy objective [36, 41]. Because generating video is costly, recent

work changes what is forecast, such as dynamic regions, depth, and semantic features [51], flow and geometry alongside RGB [47], object masks or slots [24, 45], or compact latents [33, 55], and asks whether the forecast is needed at test time. Fast-WAM retains most of the benefit of video modeling without test-time imagination [48], and PUMA and SG-WAM drop their predictors after training [13, 53]. FoveaWAM keeps this training-only use but forecasts primarily a metric 4D state rather than pixels, features, or latents, so the target's future pose is supervised explicitly instead of being left implicit.

Object-centric and geometric motion representations. Metric 3D inputs make policies more precise and sample-efficient [29, 50]. Motion has been represented by 2D point tracks [4, 40, 43], 3D flow [14, 17, 46, 49], and object keypoints or SE(3) trajectories [16, 27, 32], while rigid-body decompositions have long been used to learn scene dynamics [8, 9]. These works describe the scene with one kind of representation, such as dense flow or tracks everywhere, or a pose for each object. Target-centric predictors, including dynamic VLAs with object-level forecasts, model only the target's future [13, 16] and leave out the bodies that block or push it. DOCIR separates the robot, the target, and obstacles, but uses these distinctions only as input encodings [12]. FoveaWAM assigns precision by role: a coarse sparse 3D flow describes the whole scene, while the target, on which contact depends to within millimeters, is additionally described by its SE(3) history and forecast.

3. FoveaWAM

We formulate dynamic manipulation as generating time-indexed command chunks (Sec. 3.1) from a representation supervised, at the execution times of these commands, with the scene's sparse 3D flow and the target's SE(3) trajectory (Fig. 2). We describe the metric inputs (Sec. 3.2), this supervision (Sec. 3.3), and how an action stream reads the representation to generate the chunk (Sec. 3.4).

3.1. Problem Formulation

We study manipulation in scenes that keep moving while the policy computes. At query t, a policy π learned from demonstrations reads an RGB image I_t and a depth map D_t captured at time τ_t, together with histories $\mathcal{T}_t$ of the target pose $T(\tau) = (R(\tau), \mathbf{p}(\tau)) \in \mathrm{SE}(3)$ and $\mathcal{R}_t$ of the robot state, all in the robot base frame, and outputs a chunk of H commands

$$(\mathbf{a}_{t,0}, \dots, \mathbf{a}_{t,H-1}) \sim \pi\big(\cdot \mid I_t, D_t, \mathcal{T}_t, \mathcal{R}_t\big), \qquad (1)$$

where command j is meant for time $\tau_t + j\Delta_a$, with control period Δ_a.

Each task specifies a *target*, whose pose the robot must match within a contact or containment tolerance. The surroundings that matter form the *context*: obstacles, which the robot must clear, and drivers, which move the target. The chunk becomes available only after inference, so each executed command must account for the target and the context at its execution time rather than at τ_t.

3.2. Metric Scene and Motion Inputs

FoveaWAM encodes the target's motion precisely and its context through the current geometry: the current RGB-D frame gives the metric layout of the scene, and a short history of target poses gives the motion that one frame does not show, which also reflects how drivers have moved the target.

Current scene. FoveaWAM anchors each patch token at a metric 3D point, so that the observation shares the base frame of the forecasts and the actions. A frozen MAE ViT-B/16 [15] turns the RGB image into 196 patch tokens on a 14×14 grid, and the depth map back-projects the center of patch i to a point $\mathbf{x}_i \in \mathbb{R}^3$ in the base frame. A linear embedding of $\mathbf{x}_i$ is added to the patch token, and $\mathbf{x}_i$ drives 3D rotary position embeddings of the patch queries and keys, so that attention between patches depends on their relative 3D positions. The points $\mathbf{x}_i$ are also the source points of the scene flow (Sec. 3.3).

Target motion. Instead of past frames, from which motion must be recovered by comparing images, FoveaWAM receives the target's recent motion as a metric SE(3) state. The history spans the last 350 ms in 8 samples at 20 Hz; its 8 tokens hold the current target pose and the seven motions between consecutive samples, each with its translation, relative rotation, time interval, and finite-difference velocity and acceleration in physical units. The velocity and acceleration that decide where the target will be thus enter the model directly, at the cost of a few low-dimensional tokens; Sec. 4.3 removes this history to measure its effect. In simulation, the poses come from the simulator; on the robot, a causal tracker estimates them from the RGB-D stream.

World stream. The patch tokens, the target tokens, and 8 robot tokens with the end-effector states at the times of the target history form the 212 scene tokens. Together with 16 learnable summary tokens that aggregate them, they make up the world stream.

3.3. Role-Aware 4D Predictive Supervision

FoveaWAM supervises its representation with two geometric forecasts of different precision: the motion of the whole scene as a sparse 3D flow on the patch grid, and, because the target's pose decides contact, the full SE(3) trajectory of the target. Both are expressed in the base frame as changes from

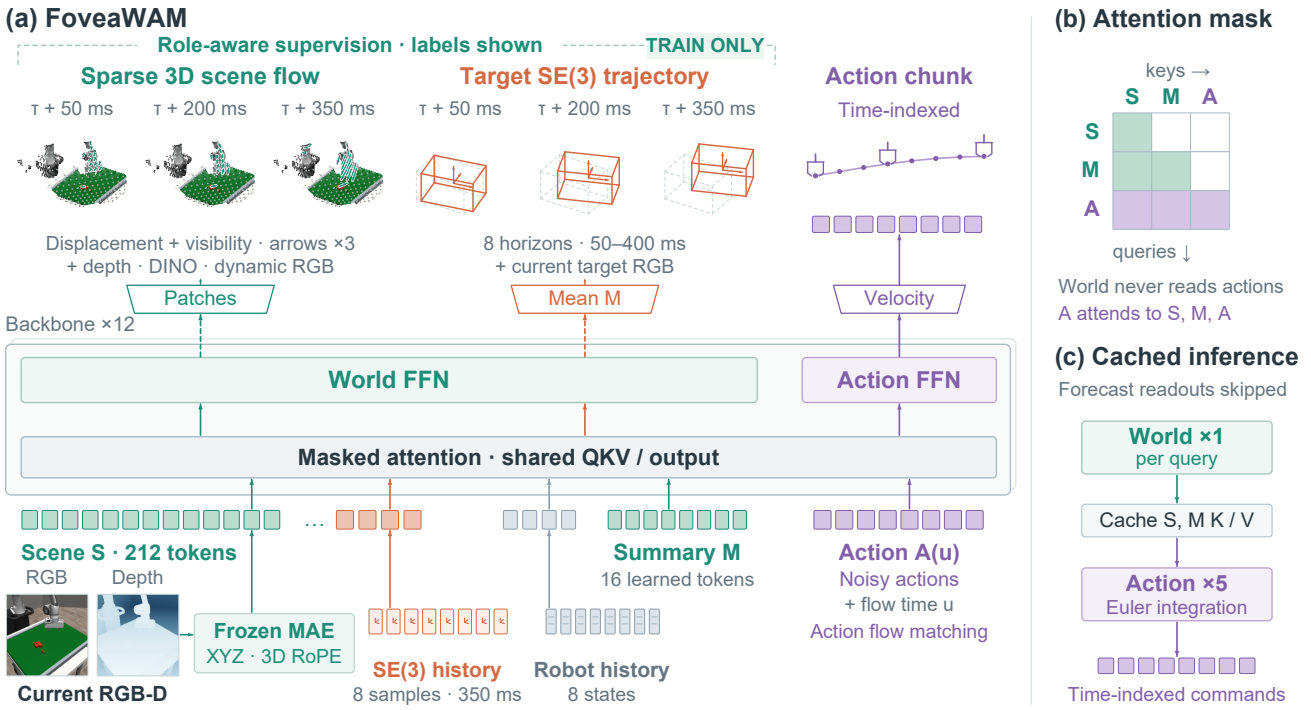


Figure 2. **FoveaWAM architecture.** (a) Read bottom to top. Current RGB-D and target/robot histories form scene tokens S; 16 learned summaries M complete the world stream. The 12-layer backbone shares attention projections and uses separate world/action FFNs. Training-only readouts (dashed) predict sparse scene displacement and visibility from patch states, and target SE(3) trajectories from the mean summary; auxiliary targets are listed below the geometric panels. The geometric examples are recorded labels at 3 of 8 horizons spanning 50–400 ms: gray glyphs mark current poses and orange glyphs future poses (glyph size illustrative); displacement arrows on the current point cloud are magnified 3×. Action icons are schematic. (b) Mask rows are queries and columns are keys: S reads S, M reads S, M, and A reads S, M, A. Actions read world hidden states. (c) Inference skips forecast readouts and caches world keys and values once per query for 5 Euler action steps.

the current state at the times $\tau_t + \delta_h$, with $\delta_h = h \cdot 50$ ms for $h = 1, \ldots, 8$, from 50 ms to 400 ms after capture; together with the current state, these horizons cover the execution times of all commands in the chunk.

Scene flow. At the resolution of the patch grid, the flow covers every surface that the patches see: the robot, the target, and the context, whose clearance and motion the commands must account for. For each of the 196 points $\mathbf{x}_i$ and each horizon h, a linear head on the final state of patch token i predicts the point's base-frame displacement $\hat{F}_{ih} \in \mathbb{R}^3$ and a visibility logit $\hat{o}_{ih}$, trained with

$$\mathcal{L}_F = \frac{1}{|\mathcal{V}_F|} \sum_{(i,h)\in\mathcal{V}_F} \Big[\tfrac{1}{3} \big\| (\hat{F}_{ih} - F_{ih})/s \big\|^2 + \mathrm{BCE}(\hat{o}_{ih}, o_{ih}) \Big], \quad (2)$$

where F_{ih} is the displacement of the surface point at $\mathbf{x}_i$, o_{ih} its visibility, s a fixed scale fitted on the training labels, and $\mathcal{V}_F$ the set of valid labels.

Target trajectory. The flow samples the target only at the few patches it covers, too sparse to determine its rotation, so a dedicated linear head on the mean of the final summary states forecasts its rigid motion directly: for each horizon h, the target's displacement $\hat{\mathbf{d}}_h$ and a rotation increment $\hat{\boldsymbol{\phi}}_h$, trained with

$$\mathcal{L}_P = \frac{1}{|\mathcal{V}|} \sum_{h\in\mathcal{V}} \Big[\|\hat{\mathbf{d}}_h - \mathbf{d}_h\|^2 + \lambda_R \, d_{\mathrm{SO}(3)}\big(\hat{R}_h, R_h\big)^2 \Big], \quad (3)$$

where $\mathbf{d}_h = \mathbf{p}(\tau_t + \delta_h) - \mathbf{p}(\tau_t)$ and $R_h = R(\tau_t + \delta_h)$ are the true displacement and orientation, $\hat{R}_h = \mathrm{Exp}(\hat{\boldsymbol{\phi}}_h^{\wedge}) R(\tau_t)$, $d_{\mathrm{SO}(3)}$ is the geodesic distance, and $\mathcal{V}$ is the set of horizons with valid labels. The rotation term matters for contact: a rotation error θ can displace a contact point at distance r from the target's origin by up to $2r \sin(\theta/2)$. For example, a 5° error at $r = 10$ cm can shift the contact point by nearly 9 mm.

3.4. World–Action Learning and Inference

Action stream. The action stream generates the chunk by conditional flow matching [22]. It holds $H = 8$ tokens, each

embedding one noisy command: an end-effector translation increment, a rotation increment, and a gripper command. In every layer, one attention spans both streams with shared projections, followed by separate feed-forward networks; world tokens use layer normalization, and action tokens use adaptive layer normalization conditioned on the flow time. With S, M, and A denoting the scene, summary, and action tokens, the attention mask allows $S \to S$, $M \to \{S, M\}$, and $A \to \{S, M, A\}$, where $X \to \mathcal{Y}$ means that tokens in X attend to tokens in $\mathcal{Y}$. The world stream never reads the actions. The action stream reads, in every layer, both the patch states shaped by scene-flow supervision and the summary states shaped by target-trajectory supervision.

Training objective. For a chunk $\mathbf{a}_1$ normalized with training statistics, noise $\mathbf{a}_0 \sim \mathcal{N}(0, I)$, and flow time $u \sim \mathcal{U}[0, 1]$, the action stream v_ψ regresses the flow-matching velocity

$$\mathcal{L}_A = \mathbb{E}\,\mathrm{MSE}\big(v_\psi(\mathbf{a}_u, u \mid \mathcal{S}_t),\, \mathbf{a}_1 - \mathbf{a}_0\big), \tag{4}$$

where $\mathbf{a}_u = (1-u)\mathbf{a}_0 + u\mathbf{a}_1$, $\mathcal{S}_t$ denotes the inputs in Eq. (1), and the MSE averages over valid commands and action dimensions. The world and action streams are trained jointly with

$$\mathcal{L} = \mathcal{L}_A + \lambda_F \mathcal{L}_F + \lambda_P \mathcal{L}_P + \lambda_{\text{aux}} \mathcal{L}_{\text{aux}}, \tag{5}$$

where $\mathcal{L}_F$ and $\mathcal{L}_P$ are the flow and pose losses of Eqs. (2) and (3), and $\mathcal{L}_{\text{aux}}$ sums auxiliary appearance losses.

Inference. At deployment, FoveaWAM skips the forecast heads, as recent world–action models do [20, 48, 51]. Because world tokens attend neither to the actions nor to the flow time, the world stream runs once per query, and the action stream integrates 5 Euler steps on the cached keys and values of the scene and summary tokens. Commands are indexed by time from capture, so the policy needs no latency input: the executor hands over each chunk at the first control-period boundary at or after inference completes and starts from the command indexed to that boundary. On the robot, the latency of a query also includes tracking the target in the newest frame. Further implementation details are in the supplementary material.

4. Experiments

We test whether role-aware precision and low latency together improve manipulation when the scene keeps moving during inference. We first compare FoveaWAM with existing policies, each charged with its own latency (Sec. 4.2), then analyze which parts of its design account for the results (Sec. 4.3), and finally verify it on a real robot with tracked target poses (Sec. 4.4).

4.1. Experimental Setup

OnTimeBench. Unlike the evaluators of DOMINO [13] and PhysMani-Bench [49], which advance the simulation only after a policy has returned its prediction, OnTimeBench shares one physical clock among observation, scene motion, and action execution, so the scene keeps moving while a policy computes. It contains fourteen tasks in five families, built in MuJoCo/robosuite [37, 57]: grasping a moving target (harmonic, conveyor, translating, rotating, and occluded motion), placing into a moving receptacle, passing a wand along a rotating wire, capturing rolling or sliding objects, and grasping after an impact (Fig. 3). Experts with privileged simulator state provide 800 demonstrations per task. We select configurations on development scenes and report all OnTimeBench results on 1,400 held-out scenes (100 per task).

Evaluation. During each query, physics advances by the method's inference latency, drawn from its latency distribution measured in its native deployment setting at batch size 1 on one RTX 5090, the GPU of our robot. All methods share one executor, which executes each chunk from the command meant for the control-period boundary at which the chunk arrives. Besides this native setting, we add 150 ms of latency to every method and, as a reference without the cost of computation, pause physics during inference. We report the task-balanced success rate (SR) and the p95 time-to-action, the measured time from an observation to its first command; paired 95% intervals come from a bootstrap over held-out scenes. The supplementary material gives the full evaluation protocol.

Baselines. We compare with the reactive policies ACT [54] and Diffusion Policy [11] and the VLA $\pi_{0.5}$ [5]; the dynamic VLAs DynamicVLA [42] and PUMA [13]; the WAM OpenWAM-α [38]; and the dynamic WAM DynamicWAM [25] (Table 1). Each baseline is trained on the same demonstrations with its official architecture and a task-adapted recipe. FoveaWAM receives a single agent-view RGB-D frame and the target's SE(3) history, without past frames. To give baselines the same target-motion information, $\pi_{0.5}$ and DynamicWAM also run with our SE(3) history tokens (+SE(3)).

Implementation. The transformer of FoveaWAM has 12 layers of width 1024 with 16 heads and 271.8M trainable parameters on top of the frozen MAE ViT-B/16 encoder. On OnTimeBench, it trains from random initialization with AdamW for at most 20 epochs, and we keep the checkpoint with the lowest validation loss. The selected seed-0 run stops after 10 epochs (25,870 updates); its best checkpoint is from epoch 8 (20,696 updates). Evaluation uses compiled BF16

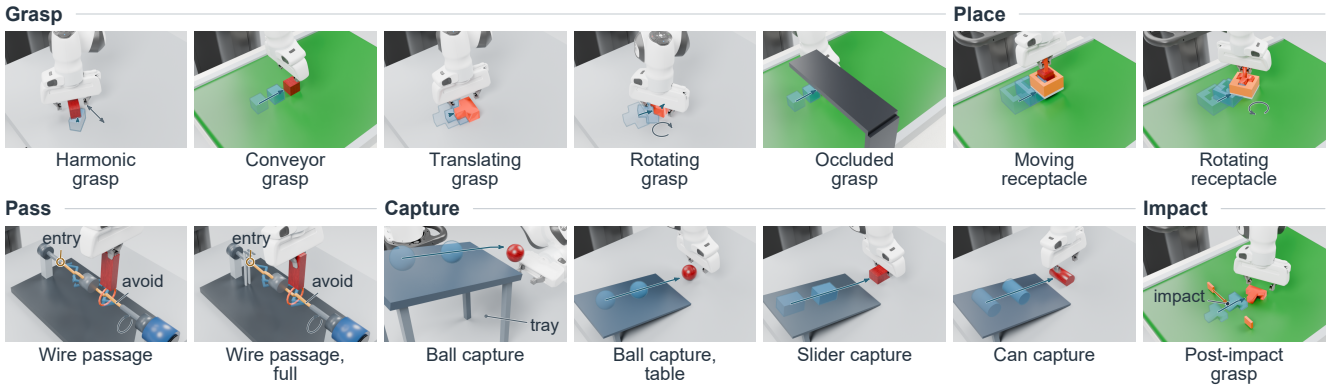


Figure 3. **OnTimeBench.** Fourteen tasks in five families, rendered from recorded simulator states with a common agent view. Translucent blue poses show selected history, fading with age; navy arrows follow recorded translational target motion. Local gray symbols denote rotation or oscillation qualitatively. An amber arrow shows the recorded striker approach; the *impact* label marks its contact location. Both Pass panels show passage: the wire is a target at entry and an obstacle along the path; Full additionally requires acquisition, exit and release. Occluded motion stays hidden. These native-expert examples illustrate task dynamics, not FoveaWAM outcomes; no future prediction is shown.

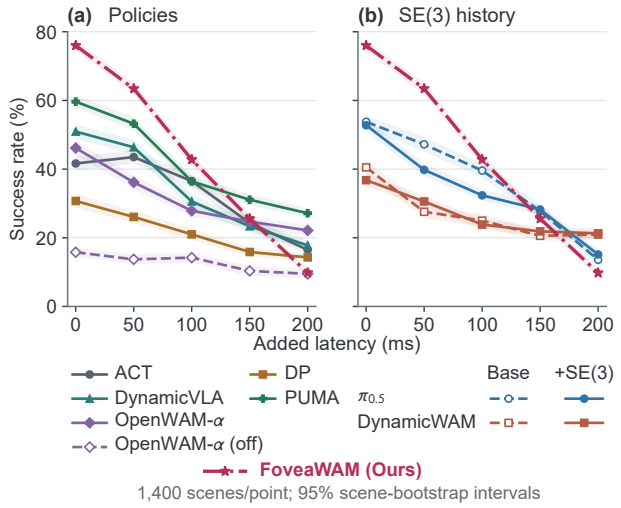


Figure 4. **Success under added latency.** Task-balanced SR on 1,400 held-out OnTimeBench scenes. Delay is added to each method's native latency. (a) Baseline deployments; off disables OpenWAM-α caching and compilation. (b) Base versus +SE(3)-history variants. PUMA uses its official three-camera D14 timing budget for the single-agentview E7 rollout. FoveaWAM uses compiled BF16 (p95 15.1 ms). Bands: 95% task-stratified scene-bootstrap intervals.

and five Euler steps with TF32 disabled. Compilation and recorded-observation warm-up finish before physics starts; history and the inference seed are then reset.

4.2. Dynamic Manipulation Results

Comparison with baselines. With its measured latency, compiled BF16 FoveaWAM reaches 76.0% SR on held-out OnTimeBench scenes (Table 1); the strongest baseline reaches 59.6%, and the strongest baseline receiving the same

Table 1. **Results on OnTimeBench.** Task-balanced SR (%) on 1,400 held-out scenes with each method's native latency, with 150 ms added, and with physics paused during inference. p95: time-to-action in each method's native deployment setting on one RTX 5090. History: inputs that carry past motion beyond the current frame. OpenWAM-α uses its official caching and compilation, and (off) disables both. PUMA's p95 uses the official three-camera D14 timing budget for its single-agentview E7 rollout; other baselines use two-camera timing inputs; FoveaWAM uses compiled BF16, one RGB-D view and simulator target poses. †For FoveaWAM, Paused uses the prior native synchronous result (1,162/1,400); the compiled-BF16 shared-executor Paused rerun is 82.0%. Best and second best per column are highlighted. FoveaWAM uses compiled BF16 inference with a measured p95 of 15.1 ms.

Method	History			SR (%) ↑			
	Wrist RGB	Flow	SE(3)	Native	+150 ms	Paused	p95 (ms) ↓
ACT [54]				41.6	24.3	40.6	**5.8**
DP [11]	✓			30.7	15.9	31.0	68.9
$\pi_{0.5}$ [5]				53.8	27.8	66.0	59.6
$\pi_{0.5}$+SE(3)			✓	52.8	28.2	74.9	61.2
DynamicVLA [42]	✓			50.9	23.4	54.6	97.3
PUMA [13]		✓		59.6	**31.1**	52.8	60.6
OpenWAM-α [38]				46.1	24.7	58.3	151.7
OpenWAM-α (off)				15.8	10.4	58.3	466.3
DynamicWAM [25]		✓		40.5	20.6	49.1	149.6
DynamicWAM+SE(3)		✓	✓	36.8	21.9	64.1	153.3
FoveaWAM			✓	**76.0**	25.6	**83.0**†	15.1

Table 2. **DOMINO results** on 24 single-arm Franka clean-L1 tasks, with the same 1,200 test scenes per method (50 per task). Intercept/Track contain 8/16 tasks; All weights all 24 tasks equally. SR and MS use a 0–100 scale and retain failures. Best and second best are highlighted.

Method	Intercept	Track	All ↑	MS ↑
$\pi_{0.5}$	**33.8**	9.1	17.3	31.1
DynamicWAM	18.3	**34.9**	**29.3**	**42.8**
PUMA	29.5	12.8	18.3	33.4
FoveaWAM	—	—	—	—

Table 3. **Real-robot results.** Success rate (%) on the six tasks of Sec. 4.4, 20 trials per task and method; All pools the trials. p95: end-to-end time-to-action (ms) on one RTX 5090, including tracking for FoveaWAM. Best per column is highlighted, including ties.

Method	T1	T2	T3	T4	T5	T6	All ↑	p95 (ms) ↓
$\pi_{0.5}$	—	—	—	—	—	—	—	—
DynamicWAM	—	—	—	—	—	—	—	—
ACT	—	—	—	—	—	—	—	—
FoveaWAM	—	—	—	—	—	—	—	—

target-motion history reaches 52.8%. Its p95 time-to-action is 15.1 ms, compared with PUMA's 60.6 ms and OpenWAM-α's 151.7 ms. Table S4 gives the Native per-task results at +0 ms extra delay; the task-balanced difference from PUMA is +16.4 points (95% paired scene-bootstrap interval [14.3, 18.5]).

Effect of inference delay. Table 1 retains the previously accepted 83.0% native synchronous result as the paused reference for FoveaWAM. The independently audited compiled-BF16 rerun with the shared executor and physics paused during inference reaches 82.0% SR. With 150 ms added to its measured inference latency, FoveaWAM reaches 25.6% SR; the strongest baseline under this condition, PUMA, reaches 31.1%. Fig. 4 shows all five measured points from 0 to 200 ms of added latency.

Results on DOMINO. To test the design beyond OnTimeBench, we train FoveaWAM, $\pi_{0.5}$, DynamicWAM, and PUMA on 24 single-arm Franka tasks of DOMINO [13] with its demonstrations and evaluate them with its native evaluator; as on OnTimeBench, FoveaWAM reads the target's simulator pose. Among the completed baselines, DynamicWAM has the highest overall SR at 29.3%, followed by PUMA at 18.3% and $\pi_{0.5}$ at 17.3% (Table 2); $\pi_{0.5}$ leads on Intercept tasks at 33.8%. [Add the accepted result and comparison for FoveaWAM.] More results are in the supplementary material.

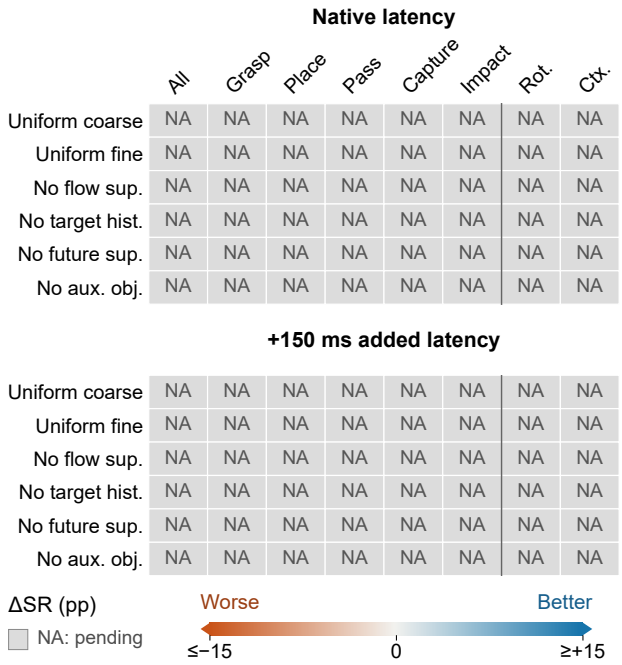


Figure 5. **Ablation effects by task (design preview; data pending).** ΔSR = SR(control) − SR(full), in percentage points; colors saturate at ±15. Rows: the six controls of Sec. 4.3; *uniform coarse flow* removes the dedicated SE(3) supervision, so that the target receives only the scene-flow supervision shared by the whole scene, and *uniform fine flow* also removes it but supervises the flow on a [56×56] grid. The full model reaches [XX.X]% SR with native latency and [XX.X]% with +150 ms. Rot./Ctx.: predeclared, overlapping rotation-sensitive/context-active groups (Table S2). Bold: the paired 95% scene-bootstrap interval excludes zero. NA: not available.

4.3. Analysis of Role-Aware Modeling

Six controls each change one part of FoveaWAM and keep its data and recipe (Fig. 5). Except for the control without target history, each changes only training losses whose heads are skipped at deployment, so it shares the inference of FoveaWAM. We report each change in SR with native latency and with 150 ms added.

Allocation of precision. Without the dedicated SE(3) supervision, the target receives only the uniform coarse scene-flow supervision shared by the whole scene, and SR drops by [X.X] and [X.X] points. Making this uniform flow fine instead, on a [56×56] grid that places [about X] points on a typical target, changes SR by [X.X] and [X.X] points, [so precision spread over the whole scene does not replace a dedicated target trajectory]. Without the scene-flow supervision, the target keeps its SE(3) trajectory while the rest of the scene, including the context, loses its motion supervision, and SR drops by [X.X] and [X.X] points. All three keep all

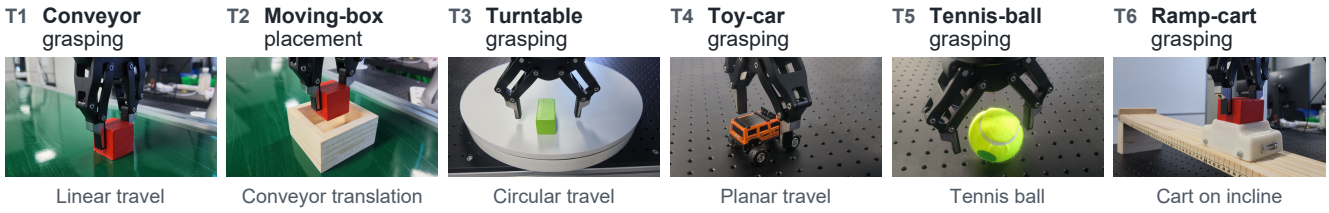


Figure 6. **Real-robot task setups.** Photographs of conveyor grasping (T1), placement into a moving box (T2), turntable grasping (T3), moving toy-car grasping (T4), tennis-ball grasping (T5), and block grasping from a ramp cart (T6).

auxiliary objectives, as the full model does. If precision follows the roles, the SE(3) supervision should matter most on the predeclared rotation-sensitive tasks, and the flow supervision on the context-active tasks, whose context constrains the robot or moves the target. [Whether Fig. 5 shows this pattern, and what it implies for the allocation of precision.]

Target motion history. Removing the past target poses with their velocity and acceleration features, while keeping the current image, the current target pose, and the robot history, lowers SR by [X.X] and [X.X] points. [The drop grows with the added latency,] as expected if the history supplies the target's velocity: a longer delay lets a velocity error accumulate further. Because the history relies on pose estimates, we also perturb them; under synthetic correlated pose noise with per-axis standard deviations of 5 mm and 1°, FoveaWAM keeps [XX.X]% SR, [X.X] points below noise-free poses.

Future supervision and auxiliary objectives. Removing all future supervision, that is, the SE(3), scene-flow, and future appearance targets, while keeping the reconstruction of the current target, leaves inference unchanged yet lowers SR by [X.X] and [X.X] points, most on [tasks with non-uniform target motion]. Removing instead the auxiliary objectives $\mathcal{L}_{\text{aux}}$, including the current-target reconstruction, and keeping both geometric forecasts changes SR by [X.X] points, [what this implies for the role of the auxiliary objectives]. [One-sentence summary of which components matter, and where.]

4.4. Real-Robot Evaluation

Setup. We evaluate six tasks on a Franka arm ([model]) with [gripper model] and a fixed RealSense D435 RGB-D camera (Fig. 6): conveyor grasping (T1), placement into a moving box (T2), turntable grasping (T3), moving toy-car grasping (T4), tennis-ball grasping (T5), and block grasping from a ramp cart (T6). Baselines are the strongest simulation baseline, $\pi_{0.5}$; a low-latency imitation policy, ACT; and a dynamic WAM, DynamicWAM. All methods train or fine-tune on the same 100 teleoperated demonstrations per task, FoveaWAM from a 48-layer base pretrained on DROID [18], and run 20 trials per task with shared trial keys. Baselines receive RGB [and a wrist camera if their recipe uses one]; FoveaWAM receives the third-person RGB-D frame and FoundationPose [39] target estimates, updated at every control-period boundary before its query.

Results. FoveaWAM succeeds in [XX.X]% of all trials, against [XX.X]% for [the strongest baseline], a paired difference of [X.X] points (95% interval [[X.X, X.X]]; Table 3). Its p95 latency, including tracking, is [XX] ms, [within one control period], whereas [name] needs [XX] ms. The margin is largest on [tasks]. [Whether the ranking of Table 1 carries over to real perception, and what tracking costs.] [One trajectory-verified contact example: how the target's pose, rotation, or context explains a success of FoveaWAM or a failure of a baseline.] The dominant failure of FoveaWAM is [failure mode]. Additional details are in the supplementary material.

5. Conclusion

FoveaWAM spends precision where contact depends on it: a metric SE(3) history and forecast for the target and a sparse 3D flow for the whole scene, with the forecasts used only as training supervision. When every method is charged its own latency, these choices let FoveaWAM act both precisely and quickly: it outperforms reactive policies, VLAs, and world–action models, including ones built for moving targets, on OnTimeBench and on six real-robot tasks at a p95 latency of 15.1 ms on OnTimeBench with compiled BF16 inference, and it also leads on DOMINO. The strongest baseline is PUMA under native latency and $\pi_{0.5}$+SE(3) in the paused column. [The one or two strongest ablation findings.] We will release OnTimeBench so that future dynamic policies are evaluated with their latency charged.

Limitations. FoveaWAM requires a specified rigid target with reliable pose estimates, and its forecast ignores how the robot's own contact changes the target's motion; learning contact-conditioned target dynamics is a natural extension. Its target-trajectory head predicts a single future per horizon and so cannot represent multimodal motion; an uncertainty-

aware forecast could address this. Our tasks contain no context that moves independently of the target; such scenes would call for a motion history of the context as well, for example of its scene flow.

LLM Usage Statement

We used OpenAI Codex to assist with literature review, code implementation and debugging, experiment execution, and language editing of the manuscript. Codex also assisted in developing plotting code and refining figure and manuscript layouts. The authors reviewed AI-assisted content, checked references against original sources, and validated the implementation through testing and experiments. The authors take full responsibility for the final manuscript, experimental results, and conclusions.

References

[1] Peter K. Allen, Aleksandar Timcenko, Billibon Yoshimi, and Paul Michelman. Automated tracking and grasping of a moving object with a robotic hand-eye system. *IEEE Transactions on Robotics and Automation*, 9(2):152–165, 1993. 2

[2] Yaakov Bar-Shalom, X. Rong Li, and Thiagalingam Kirubarajan. *Estimation with Applications to Tracking and Navigation: Theory, Algorithms and Software*. Wiley, 2001.

[3] Berthold Bäuml, Thomas Wimböck, and Gerd Hirzinger. Kinematically optimal catching a flying ball with a hand-arm-system. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2010. 2

[4] Homanga Bharadhwaj, Roozbeh Mottaghi, Abhinav Gupta, and Shubham Tulsiani. Track2Act: Predicting point tracks from internet videos enables generalizable robot manipulation. In *European Conference on Computer Vision*, 2024. 3

[5] Kevin Black, Noah Brown, James Darpinian, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In *Proceedings of The 9th Conference on Robot Learning*, pages 17–40, 2025. 1, 5, 6

[6] Kevin Black, Noah Brown, Danny Driess, et al. π_0: A vision-language-action flow model for general robot control. In *Proceedings of Robotics: Science and Systems*, 2025. 1

[7] Kevin Black, Manuel Galliker, and Sergey Levine. Real-time execution of action chunking flow policies. In *Advances in Neural Information Processing Systems*, pages 33383–33407, 2025. 2

[8] Arunkumar Byravan and Dieter Fox. SE3-Nets: Learning rigid body motion using deep neural networks. In *IEEE International Conference on Robotics and Automation*, pages 173–180, 2017. 3

[9] Arunkumar Byravan, Felix Leeb, Franziska Meier, and Dieter Fox. SE3-Pose-Nets: Structured deep dynamics models for visuomotor control. In *IEEE International Conference on Robotics and Automation*, 2018. 3

[10] Chongkei Chang and Zhidong Deng. DynaWM: A base-VLA-guided world foundation model for moving-object manipulation. *arXiv preprint arXiv:2607.02604*, 2026. 2

[11] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin C. M. Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems*, 2023. 1, 5, 6

[12] David Emukpere, Romain Deffayet, Bingbing Wu, Romain Brégier, Michael Niemaz, Jean-Luc Meunier, Denys Proux, Jean-Michel Renders, and Seungsu Kim. Disentangled object-centric image representation for robotic manipulation. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 15873–15879, 2025. 3

[13] Heng Fang, Shangru Li, Shuhan Wang, Xuanyang Xi, Dingkang Liang, and Xiang Bai. Towards generalizable robotic manipulation in dynamic environments. In *European Conference on Computer Vision*, pages 314–333, 2026. 2, 3, 5, 6, 7

[14] Jun Guo, Xiaojian Ma, Yikai Wang, Min Yang, Huaping Liu, and Qing Li. FlowDreamer: A RGB-D world model with flow-based motion representations for robot manipulation. *IEEE Robotics and Automation Letters*, 11(3):2466–2473, 2026. 3

[15] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16000–16009, 2022. 3

[16] Cheng-Chun Hsu, Bowen Wen, Jie Xu, Yashraj Narang, Xiaolong Wang, Yuke Zhu, Joydeep Biswas, and Stan Birchfield. SPOT: SE(3) pose trajectory diffusion for object-centric manipulation. In *IEEE International Conference on Robotics and Automation*, 2025. 2, 3

[17] Wenlong Huang, Yu-Wei Chao, Arsalan Mousavian, Ming-Yu Liu, Dieter Fox, Kaichun Mo, and Li Fei-Fei. PointWorld: Scaling 3D world models for in-the-wild robotic manipulation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 20765–20779, 2026. 3

[18] Alexander Khazatsky, Karl Pertsch, Suraj Nair, et al. DROID: A large-scale in-the-wild robot manipulation dataset. In *Proceedings of Robotics: Science and Systems*, 2024. 8

[19] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, et al. OpenVLA: An open-source vision-language-action model. In *Conference on Robot Learning*, pages 2679–2713, 2025. 1

[20] Shuang Li, Yihuai Gao, Dorsa Sadigh, and Shuran Song. Unified video action model. In *Proceedings of Robotics: Science and Systems*, 2025. 2, 5

[21] Haoran Liao, Pengyue Wang, Shuoyu Chen, Kehan Cheng, Xuhang Chen, Yuhao Lin, Mu Lin, Zhizhao Liang, Xiaoyi Fan, Chengyi Xing, Dan Niu, Yi-Lin Wei, and Wei-Shi Zheng. DynamicManip: Enabling dynamic manipulation from a single static demonstration. *arXiv preprint arXiv:2608.01452*, 2026. 2

[22] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023. 4

[23] Yuejiang Liu, Jubayer Hamid, Annie Xie, Yoonho Lee, Max Du, and Chelsea Finn. Bidirectional decoding: Improving action chunking via guided test-time sampling. In *International Conference on Learning Representations*, pages 4594–4627, 2025. 2

[24] Yushan Liu, Peibo Sun, Shoujie Li, Yifan Xie, Lingfeng Zhang, Xintao Chao, Shiyuan Dong, Fang Chen, Xiao-Ping

Zhang, and Wenbo Ding. OA-WAM: Object-addressable world action model for robust robot manipulation. *arXiv preprint arXiv:2605.06481*, 2026. 3

[25] Yunfan Lou, Hewen Gao, Xiyu Zhu, Zhuoran Qiao, Xuan Han, Yifan Yang, Yifan Ye, Boxian Yao, and Zhibo Pang. DynamicWAM: Dual-path motion conditioning for world-action models in dynamic manipulation. *arXiv preprint arXiv:2608.00793*, 2026. 2, 5, 6

[26] Siyuan Ma, Yutian Zhang, Boshi Zhang, Qinglian Wu, Jiaqi Zhai, Dong Wei, and Xiaojin Huang. ForeTime-VLA: Causal future-token distillation from a world action model for conveyor-belt manipulation. *arXiv preprint arXiv:2608.20735*, 2026. 2

[27] Lucas Manuelli, Wei Gao, Peter Florence, and Russ Tedrake. kPAM: Keypoint affordances for category-level robotic manipulation. In *Robotics Research: The 19th International Symposium ISRR*, pages 132–157. Springer, 2022. 3

[28] Naresh Marturi, Marek Kopicki, Alireza Rastegarpanah, et al. Dynamic grasp and trajectory planning for moving objects. *Autonomous Robots*, 43(5):1241–1256, 2019. 2

[29] Mohit Shridhar, Lucas Manuelli, and Dieter Fox. Perceiver-actor: A multi-task transformer for robotic manipulation. In *Conference on Robot Learning*, pages 785–799, 2023. 3

[30] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, et al. SmolVLA: A vision-language-action model for affordable and efficient robotics. *arXiv preprint arXiv:2506.01844*, 2025. 2

[31] Otto J. M. Smith. Closer control of loops with dead time. *Chemical Engineering Progress*, 53(5):217–219, 1957. 2

[32] Rustin Soraki, Homanga Bharadhwaj, Ali Farhadi, and Roozbeh Mottaghi. ObjectForesight: Predicting future 3D object trajectories from human videos. In *European Conference on Computer Vision*, pages 437–453, 2026. 3

[33] Yue Su, Sijin Chen, Haixin Shi, Mingyu Liu, Zhengshen Zhang, Ningyuan Huang, Weiheng Zhong, Zhengbang Zhu, Yuxiao Liu, and Xihui Liu. World guidance: World modeling in condition space for action generation. In *Proceedings of the 43rd International Conference on Machine Learning*, pages 115917–115933, 2026. 3

[34] Shahram Najam Syed, Arthur Jakobsson, Haoran Hao, and Jeffrey Ichnowski. Intercepting the future: Latent-space predictive world model for dynamic VLA manipulation. In *Conference on Robot Learning*, 2026. Accepted for publication. 2

[35] Jiaming Tang, Yufei Sun, Yilong Zhao, Shang Yang, Yujun Lin, Zhuoyang Zhang, James Hou, Yao Lu, Zhijian Liu, and Song Han. VLASH: Real-time VLAs via future-state-aware asynchronous inference. *arXiv preprint arXiv:2512.01031*, 2025. 2

[36] Yang Tian, Sizhe Yang, Jia Zeng, Ping Wang, Dahua Lin, Hao Dong, and Jiangmiao Pang. Predictive inverse dynamics models are scalable learners for robotic manipulation. In *International Conference on Learning Representations*, pages 92033–92052, 2025. 2

[37] Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033, 2012. 5

[38] Yuran Wang, Siqiao Huang, Mingleyang Li, Chenhao Zhang, Jiaqi Liang, Weiyang Jin, Yue Chen, Xuemin Chi, Donghao Zhou, Qize Yu, Yu-Kai Wang, Yuhan Rui, Shenzhe Yao, Zhen Yuan, Zhenhao Shen, Kefei Zhu, Zijie Zhu, Ning Gao, Xiaowei Chi, Guanqi He, Shanghang Zhang, Hao Dong, Lin Shao, and Hang Zhao. OpenWAM: An open, modular exploration towards systematic world-action model pretraining. *arXiv preprint arXiv:2609.07398*, 2026. 1, 2, 5, 6

[39] Bowen Wen, Wei Yang, Jan Kautz, and Stan Birchfield. FoundationPose: Unified 6D pose estimation and tracking of novel objects. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17868–17879, 2024. 8

[40] Chuan Wen, Xingyu Lin, John Ian Reyes So, Kai Chen, Qi Dou, Yang Gao, and Pieter Abbeel. Any-point trajectory modeling for policy learning. In *Proceedings of Robotics: Science and Systems*, 2024. 3

[41] Hongtao Wu, Ya Jing, Chilam Cheang, Guangzeng Chen, Jiafeng Xu, Xinghang Li, Minghuan Liu, Hang Li, and Tao Kong. Unleashing large-scale video generative pre-training for visual robot manipulation. In *International Conference on Learning Representations*, pages 10641–10662, 2024. 2

[42] Haozhe Xie, Beichen Wen, Jiarui Zheng, Zhaoxi Chen, Fangzhou Hong, Haiwen Diao, and Ziwei Liu. DynamicVLA: A vision-language-action model for dynamic object manipulation. In *Advances in Neural Information Processing Systems*, 2026. Accepted for publication. 2, 5, 6

[43] Mengda Xu, Zhenjia Xu, Yinghao Xu, Cheng Chi, Gordon Wetzstein, Manuela Veloso, and Shuran Song. Flow as the cross-domain manipulation interface. In *Conference on Robot Learning*, pages 2475–2499, 2025. 3

[44] Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, et al. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026. 1, 2

[45] Hanyang Yu, Haitao Lin, Jingbo Zhang, Wenyao Zhang, Chenghao Gu, Heng Li, and Ping Tan. MaskWAM: Unifying mask prompting and prediction for world-action models. *arXiv preprint arXiv:2606.13515*, 2026. 3

[46] Chengbo Yuan, Chuan Wen, Tong Zhang, and Yang Gao. General flow as foundation affordance for scalable robot learning. In *Conference on Robot Learning*, pages 1541–1566, 2025. 3

[47] Shanglin Yuan, Weiheng Zhao, Xin Shi, Haoyi Jiang, Xianda Guo, Liu Liu, Wenyu Liu, Wei Sui, and Xinggang Wang. DreamWAM: Beyond RGB future prediction for world action models. *arXiv preprint arXiv:2608.04996*, 2026. 3

[48] Tianyuan Yuan, Zibin Dong, Yicheng Liu, and Hang Zhao. Fast-WAM: Do world action models need test-time future imagination? In *Advances in Neural Information Processing Systems*, 2026. Accepted for publication. 2, 3, 5

[49] Peng Yun, Shouwang Huang, Hao Li, Jinxi Li, Jianan Wang, and Bo Yang. PhysMani: Physics-principled 3D world model for dynamic object manipulation. In *European Conference on Computer Vision*, pages 20–39, 2026. 2, 3, 5

[50] Yanjie Ze, Gu Zhang, Kangning Zhang, Chenyuan Hu, Muhan Wang, and Huazhe Xu. 3D diffusion policy: Generalizable visuomotor policy learning via simple 3D representations. In *Proceedings of Robotics: Science and Systems*, 2024. 3

[51] Wenyao Zhang, Hongsi Liu, Zekun Qi, Yunnan Wang, XinQiang Yu, Jiazhao Zhang, Runpei Dong, Jiawei He, He Wang, Zhizheng Zhang, Li Yi, Wenjun Zeng, and Xin Jin. DreamVLA: A vision-language-action model dreamed with comprehensive world knowledge. In *Advances in Neural Information Processing Systems*, pages 24195–24228, 2025. 2, 3, 5

[52] Yifan Zhang, Ruiping Wang, and Xilin Chen. Dynamic behavior cloning with temporal feature prediction: Enhancing robotic arm manipulation in moving object tasks. *IEEE Robotics and Automation Letters*, 10(6):5209–5216, 2025. 2

[53] Ruiteng Zhao, Zhengshen Zhang, Yue Su, Wenshuo Wang, Jiahui Li, Zhiyuan Yang, Francis E. H. Tay, Marcelo H. Ang, and Haiyue Zhu. SG-WAM: Self-guided world modeling in geometry-aware policy space. *arXiv preprint arXiv:2608.01397*, 2026. 3

[54] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of Robotics: Science and Systems*, 2023. 5, 6

[55] Ruijie Zheng, Jing Wang, Scott Reed, et al. FLARE: Robot learning with implicit world modeling. In *Proceedings of The 9th Conference on Robot Learning*, pages 3952–3971. PMLR, 2025. 3

[56] Chuning Zhu, Raymond Yu, Siyuan Feng, Benjamin Burchfiel, Paarth Shah, and Abhishek Gupta. Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets. In *Proceedings of Robotics: Science and Systems*, 2025. 2

[57] Yuke Zhu, Josiah Wong, Ajay Mandlekar, Roberto Martín-Martín, Abhishek Joshi, Kevin Lin, Abhiram Maddukuri, Soroush Nasiriany, and Yifeng Zhu. robosuite: A modular simulation framework and benchmark for robot learning. *arXiv preprint arXiv:2009.12293*, 2020. 5

[58] Brianna Zitkovich, Tianhe Yu, Sichun Xu, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183, 2023. 1